LEX PROGRAM TO COUNT IDENTIFIERS IN A C PROGRAM

1. AIM

To write and execute a Lex program that reads a C source program from a .c file and counts identifier occurrences while ignoring keywords, constants, comments, whitespace, and special symbols.

2. PSEUDOCODE

1. Start the program.
2. Open the C source file program.c in read mode.
3. If the file cannot be opened, display an error and stop.
4. Scan the source file using Lex rules.
5. Ignore comments, numeric constants, whitespace, and special symbols.
6. When an identifier pattern is matched, check whether it is a C keyword.
7. If it is not a keyword, print the identifier and increment the count.
8. After scanning ends, display the total identifier occurrence count.
9. Close the file and stop.

3. LEX PROGRAM

```
%option noyywrap

%{
#include <stdio.h>
#include <string.h>

int identifier_count = 0;

char *keywords[] = {
    "auto", "break", "case", "char", "const", "continue",
    "default", "do", "double", "else", "enum", "extern",
    "float", "for", "goto", "if", "int", "long", "register",
    "return", "short", "signed", "sizeof", "static", "struct",
    "switch", "typedef", "union", "unsigned", "void",
    "volatile", "while"
};

int is_keyword(const char *word)
{
    int i;
    for (i = 0; i < sizeof(keywords) / sizeof(keywords[0]); i++)
        if (strcmp(word, keywords[i]) == 0)
            return 1;
    return 0;
}
%}

%%
"//".*                  ;
"/*"([^*]|\*+[^*/])*\*+"/" ;
[0-9]+([.][0-9]+)?      ;
[A-Za-z_][A-Za-z0-9_]*  {
```

```
                            if (!is_keyword(yytext)) {
                                printf("Identifier: %s\n", yytext);
                                identifier_count++;
                            }
                        }
[ \t\n]+                ;
.                       ;
%%

int main(void)
{
    FILE *fp = fopen("program.c", "r");

    if (fp == NULL) {
        printf("Error: Cannot open program.c\n");
        return 1;
    }

    yyin = fp;
    yylex();
    fclose(fp);

    printf("\nTotal identifiers: %d\n", identifier_count);
    return 0;
}
```

4. SAMPLE INPUT FILE (program.c)

```
int main() {
 int a,b;
 a=10;
 b=a+5;
 return 0;
}
```

5. OUTPUT IMAGE

Illustrative expected output for the given source file. Replace it with a screenshot of the actual terminal run if required by your instructor.

```
M  /c/Users/SAIL/Downloads/lex
```

```
SAIL@SAIL601-PC17     UCRT64  /c/Users/SAIL/Downloads/lex
$ ./identifiers.exe
Identifier: main
Identifier: a
Identifier: b
Identifier: a
Identifier: b
Identifier: a
Total identifiers: 6
SAIL@SAIL601-PC17     UCRT64  /c/Users/SAIL/Downloads/lex
$
```

6. RESULT

Thus, the Lex program to read a C source file and count identifier occurrences while ignoring keywords, constants, comments, whitespace, and special symbols was prepared. For the sample program, the identifier occurrences are main, a, b, a, b, and a, giving a total of 6.